

Implementing To-Do List Application Logic

Lab Objectives

After completing this lab, the student will be able to:

- Design a data model for application data
- Implement a task creation screen
- Manage application state using `setState`
- Add and delete tasks dynamically
- Build a functional To-Do list application

Tools Required

- Flutter SDK
- Dart SDK
- Android Studio or Visual Studio Code
- Android Emulator or Physical Device
- Basic knowledge of Flutter widgets and navigation (Experiment 1)

Introduction

In this lab, students will implement the **core logic of a To-Do List application**. The app will allow users to create tasks, assign them to categories, display tasks in a list, edit existing tasks, and delete tasks with confirmation.

This experiment introduces the concept of **data modeling and dynamic UI updates using `setState`**.

LAB

Task 1: Create the Task Model and Task Categories

Objective

- Create a Dart model class to represent a task.
- Create a **global list of tasks**

Requirements

The model should include:

- title
- description
- category
- completion status

Tasks can belong to different categories such as:

- Personal
- Work
- Study
- Shopping

In **models/task.dart**

```
class Task {  
  String title;  
  String description;  
  String category;  
  bool isCompleted;  
  
  Task({  
    required this.title,  
    required this.description,  
    required this.category,  
    this.isCompleted = false,  
  });  
}  
  
List<String> categories = [  
  "Personal",  
  "Work",  
  "Study",  
  "Shopping"  
];  
  
List<Task> tasks = [];
```

Task 2: Create the Task Creation Screen

Objective

Design a screen that allows the user to create a new task.

Requirements

The screen should include:

- TextField for task title
- TextField for task description
- Dropdown for selecting category
- Button to save the task

In `screens/add_task_screen.dart`

```
○ ○ ○

import 'package:flutter/material.dart';
import '../models/task.dart';

class AddTaskScreen extends StatefulWidget {
  const AddTaskScreen({super.key});

  @override
  State<AddTaskScreen> createState() => _AddTaskScreenState();
}

class _AddTaskScreenState extends State<AddTaskScreen> {
  final TextEditingController titleController = TextEditingController();
  final TextEditingController descriptionController = TextEditingController();

  List<String> categories = [
    "Personal",
    "Work",
    "Study",
    "Shopping"
  ];

  String selectedCategory = "Personal";

  void saveTask() {
    if(titleController.text.isEmpty){
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text("Please enter a task title"))
      );
      return;
    }

    Task newTask = Task(
      title: titleController.text,
      description: descriptionController.text,
      category: selectedCategory,
    );

    /// Add task to global list
    tasks.add(newTask);

    /// Go back to listing screen
    Navigator.pop(context);
  }
}
```

```

@override
Widget build(BuildContext context) {

  return Scaffold(
    appBar: AppBar(
      title: const Text("Create Task"),
    ),

    body: Padding(
      padding: const EdgeInsets.all(16),

      child: Column(
        children: [

          /// Task Title
          TextField(
            controller: titleController,
            decoration: const InputDecoration(
              labelText: "Task Title",
              border: OutlineInputBorder(),
            ),
          ),

          const SizedBox(height: 16),

          /// Task Description
          TextField(
            controller: descriptionController,
            maxLines: 3,
            decoration: const InputDecoration(
              labelText: "Task Description",
              border: OutlineInputBorder(),
            ),
          ),

          const SizedBox(height: 16),

          /// Category Dropdown
          DropdownButtonFormField<String>({
            value: selectedCategory,
            decoration: const InputDecoration(
              labelText: "Category",
              border: OutlineInputBorder(),
            ),
            items: categories.map((category) {
              return DropdownMenuItem(
                value: category,
                child: Text(category),
              );
            }).toList(),
            onChanged: (value) {
              setState(() {
                selectedCategory = value!;
              });
            },
          ),

          const SizedBox(height: 24),

          /// Add Task Button
          SizedBox(
            width: double.infinity,
            child: ElevatedButton(
              onPressed: saveTask,
              child: const Text("Add Task"),
            ),
          ),

        ],
      ),
    ),
  );
}

```

Task 3: Display Tasks in the Listing Screen

Objective

Display tasks in the existing List Screen created in Experiment 1.

Use `ListView.builder` to display tasks dynamically.

In `screens/task_list_screen.dart`

```
○ ○ ○

import 'package:flutter/material.dart';
import '../models/task.dart';
import 'add_task_screen.dart';

class TaskListScreen extends StatefulWidget {
  const TaskListScreen({super.key});

  @override
  State<TaskListScreen> createState() => _TaskListScreenState();
}

class _TaskListScreenState extends State<TaskListScreen> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("My Tasks"),
      ),
      body: ListView.builder(
        itemCount: tasks.length,
        itemBuilder: (context, index) {
          final task = tasks[index];

          return ListTile(
            title: Text(task.title),
            subtitle: Text(task.category),
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        child: const Icon(Icons.add),
        onPressed: () async {
          await Navigator.push(
            context,
            MaterialPageRoute(
              builder: (_) => const AddTaskScreen(),
            ),
          );

          /// Refresh screen
          setState(() {});
        },
      ),
    );
  }
}
```

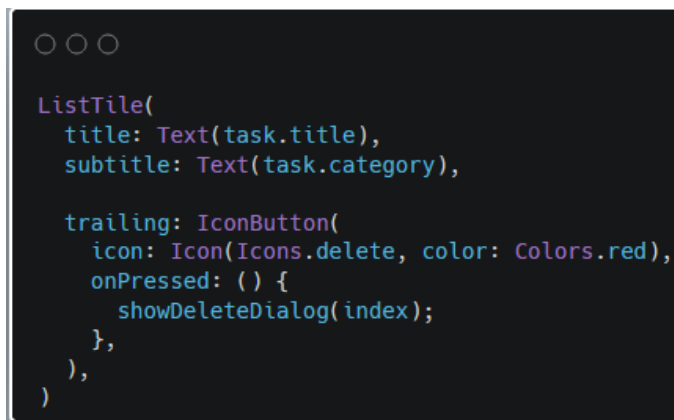
Task 4: Delete a Task with Confirmation Dialog

Objective

- Allow the user to delete a task from the task list by pressing a delete button.
- Before deleting the task, the application should display a confirmation dialog to prevent accidental deletion.

Step 1: Add a Delete Button to the ListTile

Modify the `ListTile` widget in the **Task Listing Screen** to include a delete button using the `trailing` property.



```
○ ○ ○  
  
ListTile(  
  title: Text(task.title),  
  subtitle: Text(task.category),  
  
  trailing: IconButton(  
    icon: Icon(Icons.delete, color: Colors.red),  
    onPressed: () {  
      showDeleteDialog(index);  
    },  
  ),  
)
```

Explanation

- `trailing` adds a widget to the **right side** of the `ListTile`
- `IconButton` displays a delete icon
- When the button is pressed, it calls `showDeleteDialog(index)`

Step 2: Create the Delete Confirmation Dialog

Add the following method inside the `TaskListScreen` class.

```

void showDeleteDialog(int index) {
  showDialog(
    context: context,
    builder: (context) {
      return AlertDialog(
        title: Text("Delete Task"),
        content: Text("Are you sure you want to delete this task?"),
        actions: [

          TextButton(
            onPressed: () {
              Navigator.pop(context);
            },
            child: Text("Cancel"),
          ),

          TextButton(
            onPressed: () {

              setState(() {
                tasks.removeAt(index);
              });

              Navigator.pop(context);
            },
            child: Text("Delete"),
          ),
        ],
      );
    },
  );
}

```

POST-LAB

1. Add a FloatingActionButton to create new tasks.
2. Add task priority levels (Low / Medium / High).